

Cooperative Society Membership and
Savings Management System



Name : S. Pranathi

Roll No : 25WU0104024

Course : Database Management System (DBMS)

Faculty Name : Kiram Mayee

Academic Year : 2026 – 29

TABLE OF CONTENTS

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2. Abstract

The **Cooperative Society Membership and Savings Management System** is a database-based application developed to efficiently manage the records and financial activities of a cooperative society.

The system provides a centralized platform for storing and managing member information, membership details, shares, savings deposits and withdrawals, loans, repayments, dividends, meetings, attendance, and nominee information.

It reduces the difficulties associated with manual record keeping, such as data duplication, calculation errors, loss of records, and difficulty in retrieving information. The system uses a structured relational database to maintain accurate, secure, and easily accessible information.

It also supports essential database operations such as inserting, updating, deleting, and viewing records. The proposed system helps cooperative society administrators manage member and financial records more efficiently, improves data accuracy, and provides faster access to important information.

Overall, the system offers a simple and organized solution for effective cooperative society membership and savings management.

The system comprehensively tracks and manages **cooperative society members, savings accounts, share allocations, loan applications, dividend distributions, physical or virtual meetings, and nominee details.**

By replacing traditional record-keeping with a secure, centralized database, the platform eliminates computational errors, reduces administrative workload, and provides real-time financial reporting.

Ultimately, this system secures member data, simplifies financial tracking, and fosters greater trust between the cooperative society and its members.

3. Introduction and Problem Statement

3.1 Introduction

A **cooperative society** is an organization formed by a group of people who voluntarily come together to achieve common financial and social objectives through mutual cooperation. Members of a cooperative society may contribute shares and savings and can access services such as loans and other financial benefits. The society needs to maintain various types of information, including member details, membership records, savings, shares, loans, repayments, dividends, meetings, attendance, and nominee details. Proper management of this information is essential for the smooth and transparent functioning of the society.

The **Cooperative Society Membership and Savings Management System** is designed to provide a centralized and organized method for managing these records. The system uses a relational database to store and maintain information efficiently. It allows users to add, update, delete, and view records whenever required. By connecting related information through database tables, the system makes it easier to retrieve member and financial information and reduces the effort required for maintaining records manually.

3.2 Problem Statement

To overcome these limitations, a **database management system** is required to store and organize all cooperative society information in a centralized and structured manner.

- The **Cooperative Society Membership and Savings Management System** provides a systematic solution for managing member and financial records using a relational database.

- The system allows authorized users to insert, update, delete, and retrieve records efficiently.
- It maintains relationships between members, savings, shares, loans, repayments, dividends, meetings, attendance, and nominees, making information easier to access and manage.
- By reducing manual work and improving data accuracy, consistency, and accessibility, the proposed system helps the cooperative society operate more efficiently and provides reliable information for administrative and financial decision-making.

Traditionally, cooperative societies may maintain member and financial records using paper registers, spreadsheets, or separate files. Managing a large number of records through these methods can be difficult and time-consuming. Manual record keeping may result in problems such as duplicate entries, incorrect data, calculation errors, missing records, difficulty in searching for information, and delays in updating transactions. It can also be challenging to track a member's savings, shares, loans, repayments, dividends, and other activities in an organized manner.

4. Objectives and Scope

4.1 Objectives

The main objective of the **Cooperative Society Membership and Savings Management System** is to develop a centralized database that helps manage the important activities and records of a cooperative society efficiently. The major objectives of the system are:

- **Maintain Member Information:** Store and manage member details such as member ID, name, contact information, address, and membership date.
- **Manage Savings:** Record members' savings deposits and withdrawals and maintain accurate savings information.
- **Manage Shares:** Maintain information about shares purchased or held by each member.
- **Manage Loans:** Store loan details, including loan amount, loan date, status, and the member associated with the loan.
- **Track Repayments:** Record loan repayment transactions and track the repayment history of members.
- **Manage Dividends:** Maintain dividend information and manage dividend amounts associated with members.
- **Maintain Meeting Records:** Store information about society meetings and maintain member attendance records.
- **Manage Nominees:** Maintain nominee details associated with each cooperative society member.
- **Improve Data Management:** Reduce manual work, minimize duplicate records and errors, and provide faster access to required information.

4.2 Scope

The scope of the **Cooperative Society Membership and Savings Management System** covers the storage and management of important membership and financial information within a centralized relational database.

- The system can manage member registration, member details, shares, savings deposits and withdrawals, loans, loan repayments, dividends, meetings, attendance, and nominee information.
- Authorized users can perform basic database operations such as inserting, updating, deleting, and viewing records.
- The system can also generate useful information through SQL queries, such as member details, savings information, loan records, repayment details, and other financial information.
- However, the current system is mainly designed for **database management and basic administrative operations**.
- It does not cover advanced banking functions, online payment processing, real-time integration with external banking systems, automated loan approval, mobile banking, or complex financial accounting.
- These features can be added as future enhancements. The present system focuses on providing a simple, organized, and reliable solution for managing cooperative society membership and savings-related records.

5. Software and Hardware Requirements

5.1 Software Requirements

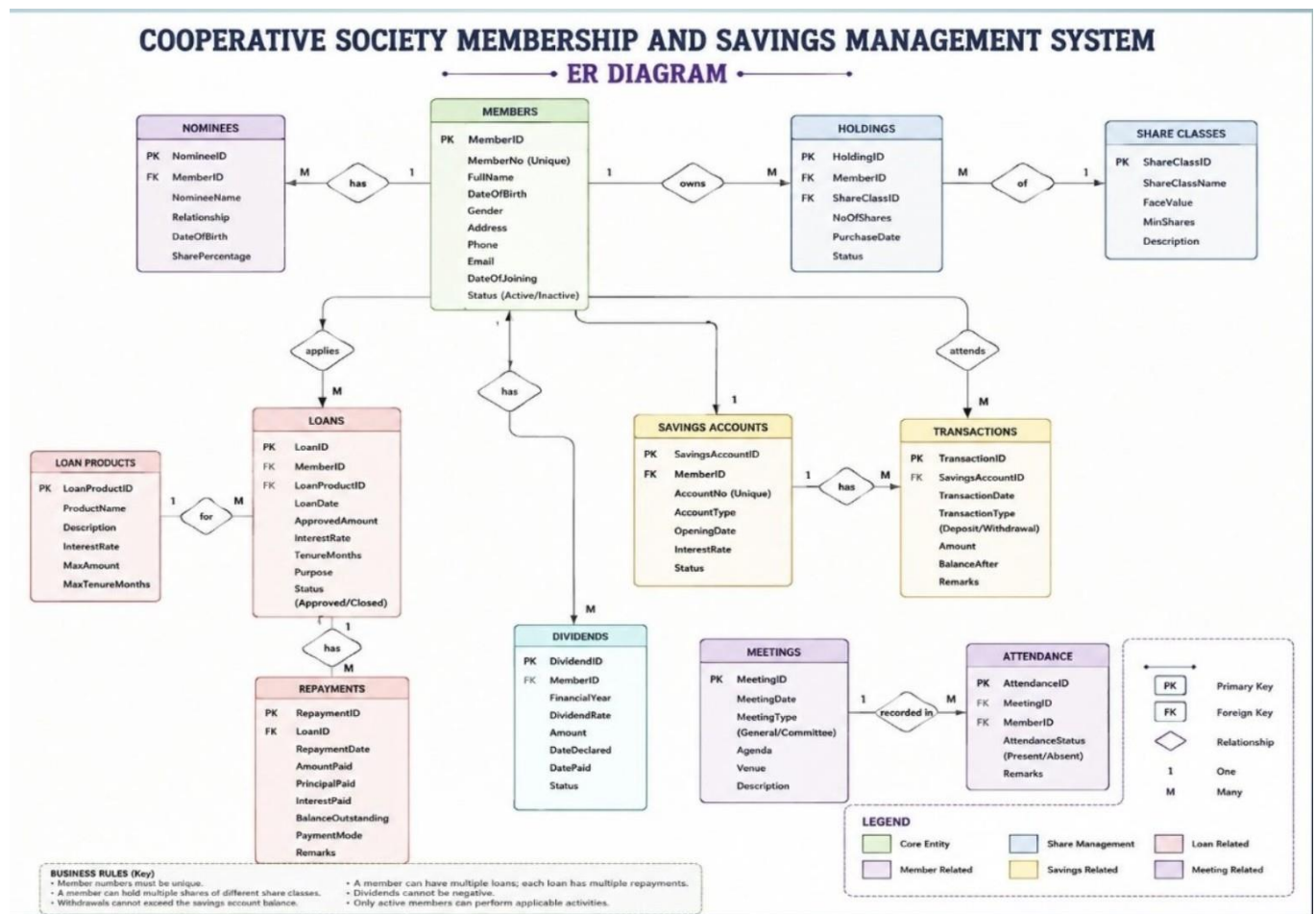
S. No.	Software / Tool	Purpose
1	MySQL	Stores and manages the project database
2	MySQL Workbench	Creates tables, executes SQL queries, and manages the database
3	Python	Develops the application and user interface
4	VS Code	Used for writing and managing source code
5	MySQL Connector	Connects the Python application with MySQL
6	GitHub	Stores and manages project files and source code

5.2 Hardware Requirements

S. No.	Hardware	Requirement
1	Computer/Laptop	Required for developing and running the project
2	Processor	Minimum dual-core processor
3	RAM	Minimum 4 GB; 8 GB recommended
4	Storage	At least 10 GB of free space
5	Internet Connection	Required for GitHub and downloading required software
6	Keyboard & Mouse	Required for entering data and operating the application
7	Display/Monitor	Required for viewing and demonstrating the application

6. ER Diagram

For the **Cooperative Society Membership and Savings Management System**, the ER (Entity–Relationship) diagram represents the main entities in the system and the relationships between them.



*Fig 6.1 : ER diagram of the Cooperative Society Membership
and Savings Management System*

7. Relational Schema and Normalization

7.1 Relational Schema

The relational schema represents the database in the form of tables. Each table contains attributes (columns), and primary and foreign keys are used to establish relationships between the tables.

MEMBER(Member_ID, Name, Phone, Address, Join_Date)

SHARE(Share_ID, Member_ID, Share_Count, Share_Value, Purchase_Date)

SAVINGS(Savings_ID, Member_ID, Amount, Transaction_Type, Transaction_Date)

LOAN(Loan_ID, Member_ID, Loan_Amount, Loan_Date, Status)

REPAYMENT(Repayment_ID, Loan_ID, Amount, Payment_Date)

DIVIDEND(Dividend_ID, Member_ID, Amount, Dividend_Year)

MEETING(Meeting_ID, Meeting_Date, Meeting_Type, Description)

ATTENDANCE(Attendance_ID, Member_ID, Meeting_ID, Status)

NOMINEE(Nominee_ID, Member_ID, Nominee_Name, Relationship, Phone)

7.2 Primary and Foreign Keys

Table	Primary Key	Foreign Key
MEMBER	Member_ID	—
SHARE	Share_ID	Member_ID
SAVINGS	Savings_ID	Member_ID
LOAN	Loan_ID	Member_ID
REPAYMENT	Repayment_ID	Loan_ID
DIVIDEND	Dividend_ID	Member_ID
MEETING	Meeting_ID	—
ATTENDANCE	Attendance_ID	Member_ID, Meeting_ID
NOMINEE	Nominee_ID	Member_ID

7.3 Normalization

Normalization is the process of organizing the database into separate, well-structured tables. It reduces **data redundancy**, prevents **data inconsistency**, and makes the database easier to maintain.

1NF — First Normal Form

A table is in **First Normal Form (1NF)** when every column contains only a **single, atomic value** and there are no repeating groups.

Example before 1NF:

Member_ID	Name	Savings
101	Ravi	₹5,000, ₹3,000, ₹2,000

The Savings column contains multiple values.

After 1NF:

Savings_ID	Member_ID	Amount
1	101	₹5,000
2	101	₹3,000
3	101	₹2,000

Each field now contains a single value, so the table satisfies **1NF**.

2NF — Second Normal Form

A table is in **Second Normal Form (2NF)** when:

- It is already in 1NF.
- Every non-key attribute depends on the **complete primary key**.

For example, if member and savings information are unnecessarily stored together, member details may be repeated for every savings transaction.

Instead, we separate them:

MEMBER

Member_ID	Name	Phone
101	Ravi	9876543210
102	Priya	9876543211

SAVINGS

Savings_ID	Member_ID	Amount
1	101	5000
2	101	3000
3	102	4000

Member information is stored only in the MEMBER table, while savings information is stored in the SAVINGS table. This removes partial dependency and satisfies **2NF**.

3NF — Third Normal Form

A table is in **Third Normal Form (3NF)** when:

- It is already in 2NF.
- There is no **transitive dependency**.
- Non-key attributes depend directly on the primary key.

For example, suppose the MEMBER table contains:

Member_ID	Name	City	City_Code
101	Ravi	Hyderabad	500001

Here, **City_Code** depends on **City**, rather than directly on **Member_ID**. To remove this dependency, city information can be maintained separately.

MEMBER

Member_ID	Name	City_ID
101	Ravi	1

City

City_ID	City	City_Code
1	Hyderabad	500001

Now the attributes depend on their respective primary keys, satisfying **3NF**.

Final Result

Therefore, the database is organized into separate related tables such as **MEMBER, SHARE, SAVINGS, LOAN, REPAYMENT, DIVIDEND, MEETING, ATTENDANCE, and NOMINEE**. Applying **1NF, 2NF, and 3NF** reduces duplicate data, improves consistency, and makes the Cooperative Society Membership and Savings Management System easier to manage.

8. Data Dictionary

A **data dictionary** provides detailed information about the fields used in the database, including their data types, constraints, and purpose.

8.1 MEMBER Table

Field	Data Type	Constraint	Description
Member_ID	INT	Primary Key	Unique identification number of the member
Name	VARCHAR(100)	NOT NULL	Full name of the member
Phone	VARCHAR(15)	UNIQUE, NOT NULL	Member's contact number
Address	VARCHAR(200)	—	Member's residential address
Join_Date	DATE	NOT NULL	Date on which the member joined the society

8.2 SHARE Table

Field	Data Type	Constraint	Description
Share_ID	INT	Primary Key	Unique identification number of the share record
Member_ID	INT	Foreign Key	Identifies the member who owns the share
Share_Count	INT	NOT NULL	Number of shares owned by the member
Share_Value	DECIMAL(10,2)	NOT NULL	Monetary value of the shares
Purchase_Date	DATE	NOT NULL	Date on which the shares were purchased

8.3.SAVINGS Table

Field	Data Type	Constraint	Description
Savings_ID	INT	Primary Key	Unique identification number of the savings transaction
Member_ID	INT	Foreign Key	Identifies the member associated with the transaction
Amount	DECIMAL(10,2)	NOT NULL	Amount deposited or withdrawn
Transaction_Type	VARCHAR(20)	NOT NULL	Specifies Deposit or Withdrawal
Transaction_Date	DATE	NOT NULL	Date of the savings transaction

8.4 LOAN Table

Field	Data Type	Constraint	Description
Loan_ID	INT	Primary Key	Unique identification number of the loan
Member_ID	INT	Foreign Key	Identifies the member who received the loan
Loan_Amount	DECIMAL(12,2)	NOT NULL	Amount of loan issued
Loan_Date	DATE	NOT NULL	Date on which the loan was issued
Status	VARCHAR(20)	NOT NULL	Current status of the loan

8.5 REPAYMENT Table

Field	Data Type	Constraint	Description
Repayment_ID	INT	Primary Key	Unique identification number of the repayment
Loan_ID	INT	Foreign Key	Identifies the loan for which repayment is made
Amount	DECIMAL(10,2)	NOT NULL	Amount repaid by the member
Payment_Date	DATE	NOT NULL	Date on which the repayment was made

8.6 DIVIDEND Table

Field	Data Type	Constraint	Description
Dividend_ID	INT	Primary Key	Unique identification number of the dividend record
Member_ID	INT	Foreign Key	Identifies the member receiving the dividend
Amount	DECIMAL(10,2)	NOT NULL	Dividend amount given to the member
Dividend_Year	YEAR	NOT NULL	Year for which the dividend is provided

8.7 MEETING Table

Field	Data Type	Constraint	Description
-------	-----------	------------	-------------

Meeting_ID	INT	Primary Key	Unique identification number of the meeting
Meeting_Date	DATE	NOT NULL	Date of the society meeting
Meeting_Type	VARCHAR(50)	NOT NULL	Type or category of the meeting
Description	VARCHAR(255)	—	Details or agenda of the meeting

8.8 ATTENDANCE Table

Field	Data Type	Constraint	Description
Attendance_ID	INT	Primary Key	Unique attendance record ID
Member_ID	INT	Foreign Key	Identifies the member attending the meeting
Meeting_ID	INT	Foreign Key	Identifies the meeting
Status	VARCHAR(20)	NOT NULL	Attendance status such as Present or Absent

8.9 NOMINEE Table

Field	Data Type	Constraint	Description
Nominee_ID	INT	Primary Key	Unique identification number of the nominee
Member_ID	INT	Foreign Key	Identifies the member associated with the nominee
Nominee_Name	VARCHAR(100)	NOT NULL	Full name of the nominee
Relationship	VARCHAR(50)	NOT NULL	Relationship of nominee with the member
Phone	VARCHAR(15)	—	Contact number of the nominee

Key Constraints Used

Constraint	Purpose
PRIMARY KEY (PK)	Uniquely identifies each record in a table
FOREIGN KEY (FK)	Creates a relationship between two tables
NOT NULL	Ensures that a field cannot be left empty
UNIQUE	Prevents duplicate values in a column

9. SQL Commands Used

SQL (Structured Query Language) is used to create, modify, and manage the database of the Cooperative Society Membership and Savings Management System. The main SQL commands used in the project are DDL (Data Definition Language) and DML (Data Manipulation Language).

9.1 DDL – Data Definition Language

DDL commands are used to create and modify the structure of the database and its tables.

1. CREATE DATABASE

The CREATE DATABASE command is used to create a new database for the project.

```
CREATE DATABASE CooperativeSocietyDB;
```

```
USE CooperativeSocietyDB;
```

Sample Output:

Query OK, 1 row affected

Database changed

2. CREATE TABLE

The CREATE TABLE command is used to create tables with columns, data types, and constraints.

Example: MEMBER Table

```
CREATE TABLE MEMBER (  
    Member_ID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15) UNIQUE NOT NULL,  
    Address VARCHAR(200),  
    Join_Date DATE NOT NULL  
);
```

Example: SAVINGS Table

```
CREATE TABLE SAVINGS (  
    Savings_ID INT PRIMARY KEY,  
    Member_ID INT NOT NULL,  
    Amount DECIMAL(10,2) NOT NULL,  
    Transaction_Type VARCHAR(20) NOT NULL,  
    Transaction_Date DATE NOT NULL,  
    FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID)  
);
```

Sample Output:

Query OK, 0 rows affected

3. ALTER TABLE

The ALTER TABLE command is used to modify the structure of an existing table.

```
ALTER TABLE MEMBER  
ADD Email VARCHAR(100);
```

Sample Output:

Query OK, 0 rows affected

Records: 0 Duplicates: 0 Warnings: 0

The Email column is now added to the MEMBER table.

9.2 DML – Data Manipulation Language

DML commands are used to insert, update, and delete records stored in database tables.

1. INSERT

The INSERT command is used to add new records to a table.

```
INSERT INTO MEMBER
(Member_ID, Name, Phone, Address, Join_Date, Email)
VALUES
(101, 'Ananya', '9876543210', 'Hyderabad', '2026-06-10',
'ananya@gmail.com');
```

Another record can be inserted as:

```
INSERT INTO MEMBER
(Member_ID, Name, Phone, Address, Join_Date, Email)
VALUES
(102, 'Sufiyan', '9876543211', 'Hyderabad', '2026-06-12',
'sufiyan@gmail.com');
```

Sample Output:

Query OK, 1 row affected

To display the inserted records:

```
SELECT * FROM MEMBER;
```

Sample Output:

Member_ID	Name	Phone	Address	Join_Date	Email
101	Ananya	9876543210	Hyderabad	2026-06-10	ananya@gmail.com
102	Sufiyan	9876543211	Hyderabad	2026-06-12	sufiyan@gmail.com

2. UPDATE

The UPDATE command is used to modify existing records.

```
UPDATE MEMBER
SET Address = 'Gachibowli'
WHERE Member_ID = 101;
```

Sample Output:

```
Query OK, 1 row affected
Rows matched: 1
Changed: 1
Warnings: 0
```

To verify the updated record:

```
SELECT * FROM MEMBER
WHERE Member_ID = 101;
```

Sample Output:

Member_ID	Name	Phone	Address	Join_Date	Email
101	Ananya	9876543210	Gachibowli	2026-06-10	ananya@gmail.com

3. DELETE

The DELETE command is used to remove an existing record from a table.

```
DELETE FROM MEMBER
WHERE Member_ID = 102;
```

Sample Output:

```
Query OK, 1 row affected
```

To verify the remaining records:

```
SELECT * FROM MEMBER;
```

Sample Output:

Member_ID	Name	Phone	Address	Join_Date	Email
101	Ananya	9876543210	Gachibowli	2026-06-10	ananya@gmail.com

Summary

DDL commands such as CREATE DATABASE, CREATE TABLE, and ALTER TABLE define and modify the database structure. DML commands such as INSERT, UPDATE, and DELETE manage the actual records stored in the database. These commands form the basic SQL operations required for managing the Cooperative Society Membership and Savings Management System.

10. Queries with Outputs

SQL queries are used to retrieve, filter, calculate, and combine information stored in the Cooperative Society Membership and Savings Management System. The following ten queries demonstrate important database operations such as SELECT, WHERE, aggregate functions, and JOIN.

Sample Data Used

The following sample records are considered for demonstrating the queries.

MEMBER Table

Member_ID	Name	Phone	Address	Join_Date
101	Ananya	9876543210	Hyderabad	2026-06-10

102	Sufiyan	9876543211	Hyderabad	2026-06-12
103	Pranathi	9876543212	Secunderabad	2026-06-15
104	Rishil	9876543213	Gachibowli	2026-06-18

SAVINGS Table

Savings_ID	Member_ID	Amount	Transaction_Type	Transaction_Date
1	101	5000.00	Deposit	2026-07-01
2	102	7500.00	Deposit	2026-07-03
3	103	3000.00	Deposit	2026-07-05
4	104	10000.00	Deposit	2026-07-08

LOAN Table

Loan_ID	Member_ID	Loan_Amount	Loan_Date	Status
201	101	50000.00	2026-07-10	Approved
202	102	75000.00	2026-07-12	Outstanding
203	103	30000.00	2026-07-15	Closed

REPAYMENT Table

Repayment_ID	Loan_ID	Amount	Payment_Date
1	201	10000.00	2026-08-01
2	202	15000.00	2026-08-03
3	203	30000.00	2026-08-05

DIVIDEND Table

Dividend_ID	Member_ID	Amount	Dividend_Year
1	101	1500.00	2026
2	102	2000.00	2026
3	103	1000.00	2026
4	104	2500.00	2026

Query 1: Display All Members

This query displays all the members registered in the cooperative society.

```
SELECT * FROM MEMBER;
```

Output:

Member_ID	Name	Phone	Address	Join_Date
101	Ananya	9876543210	Hyderabad	2026-06-10
102	Sufiyan	9876543211	Hyderabad	2026-06-12
103	Pranathi	9876543212	Secunderabad	2026-06-15
104	Rishil	9876543213	Gachibowli	2026-06-18

Query 2: Find Members from a Particular Location

This query displays members whose address is Hyderabad.

```
SELECT Member_ID, Name, Phone, Address
FROM MEMBER
WHERE Address = 'Hyderabad';
```

Output:

Member_ID	Name	Phone	Address
101	Aasha	9876543210	Hyderabad
102	Kiran	9876543211	Hyderabad

Query 3: Display Savings Records

This query displays all savings transactions recorded in the system.

```
SELECT * FROM SAVINGS;
```

Output:

Savings_ID	Member_ID	Amount	Transaction_Type	Transaction_Date
1	101	5000.00	Deposit	2026-07-01
2	102	7500.00	Deposit	2026-07-03
3	103	3000.00	Deposit	2026-07-05
4	104	10000.00	Deposit	2026-07-08

Query 4: Find Members with Savings Above ₹5,000

This query identifies savings transactions where the amount is greater than ₹5,000.

```
SELECT Savings_ID, Member_ID, Amount
FROM SAVINGS
WHERE Amount > 5000;
```

Output:

Savings_ID	Member_ID	Amount
2	102	7500.00
4	104	10000.00

Query 5: Calculate Total Savings

The SUM() function is used to calculate the total amount of savings recorded in the system.

```
SELECT SUM(Amount) AS Total_Savings
FROM SAVINGS
WHERE Transaction_Type = 'Deposit';
```

Output:

```
Total_Savings
25500.00
```

Therefore, the total savings deposited by members is **₹25,500**.

Query 6: Display Loan Details

This query displays all loan records along with their current status.

```
SELECT Loan_ID, Member_ID, Loan_Amount, Loan_Date, Status
FROM LOAN;
```

Output:

Loan_ID	Member_ID	Loan_Amount	Loan_Date	Status
201	101	50000.00	2026-07-10	Approved
202	102	75000.00	2026-07-12	Outstanding
203	103	30000.00	2026-07-15	Closed

Query 7: Find Outstanding Loans

This query displays loans that are currently outstanding.

```
SELECT Loan_ID, Member_ID, Loan_Amount, Loan_Date
FROM LOAN
WHERE Status = 'Outstanding';
```

Output:

Loan_ID	Member_ID	Loan_Amount	Loan_Date
202	102	75000.00	2026-07-12

Query 8: Display Repayment Information

This query displays all repayment transactions made against loans.

```
SELECT * FROM REPAYMENT;
```

Output:

Repayment_ID	Loan_ID	Amount	Payment_Date
1	201	10000.00	2026-08-01
2	202	15000.00	2026-08-03
3	203	30000.00	2026-08-05

Query 9: Calculate Total Dividend

The SUM() function is used to calculate the total dividend distributed to members.

```
SELECT SUM(Amount) AS Total_Dividend
FROM DIVIDEND;
```

Output:

Total_Dividend
7000.00

Therefore, the total dividend amount is ₹7,000.

Query 10: Display Member and Savings Details Using JOIN

This query combines the MEMBER and SAVINGS tables using the common Member_ID field. It displays the member name along with their savings amount.

```
SELECT
    MEMBER.Member_ID,
    MEMBER.Name,
    SAVINGS.Amount,
    SAVINGS.Transaction_Type,
    SAVINGS.Transaction_Date
FROM MEMBER
JOIN SAVINGS
ON MEMBER.Member_ID = SAVINGS.Member_ID;
```

Output:

Member_ID	Name	Amount	Transaction_Type	Transaction_Date
101	Ananya	5000.00	Deposit	2026-07-01
102	Sufiyan	7500.00	Deposit	2026-07-03

103	Pranathi	3000.00	Deposit	2026-07-05
104	Rishil	10000.00	Deposit	2026-07-08

Conclusion of Queries

The above queries demonstrate how SQL can be used to retrieve and analyze cooperative society data. Simple **SELECT** and **WHERE** queries retrieve specific records, aggregate functions such as **SUM()** calculate financial totals, and the **JOIN** operation combines information from related tables. These queries help administrators efficiently monitor members, savings, loans, repayments, and dividends.